

Министерство образования и науки РФ
Федеральное государственное автономное образовательное учреждение высшего
образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем

Кафедра Прикладная математика и фундаментальная информатика

Расчетно-графическая работа

по дисциплине Алгоритмы и анализ сложности

на тему Анализ сложности работы алгоритмов

Пояснительная записка

Шифр проекта 020-РГР-02.03.02-№ 37-ПЗ

Студента Рау Алексея Евгеньевича

фамилия, имя, отчество полностью

Курс 3

Группа ФИТ-231

Направление (специальность)

02.03.02

Фундаментальная информатика и информационные технологии

код, наименование

Руководитель

ассистент

ученая степень, звание

Зубова Ю.И.

фамилия, инициалы

Выполнил

29.05.2026

дата, подпись студента

Проверил

04.06.2026

дата, подпись руководителя

Содержание

Задание 1.....	3
Задание 2.....	6
Задание 3.....	11
Задание 4.....	20
Задание 5.....	23
Задание 6.....	26
Задание 7.....	29
Задание 8.....	32
Задание 9.....	35
Задание 10.....	39

Задание 1

Необходимо вычислить n -й член последовательности Фибоначчи двумя методами:

- 1) Рекурсивным;
- 2) Итерационным.

Код реализации:

```
import time
import matplotlib.pyplot as plt

def fibonacci_recursive(n):
    if n ≤ 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

def fibonacci_iterative(n):
    if n ≤ 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def measure_time(func, n):
    start_time = time.perf_counter()
    result = func(n)
    end_time = time.perf_counter()
    return result, (end_time - start_time) * 1000

max_n = 40
recursive_times = []
iterative_times = []
n_values = list(range(1, max_n + 1))

print("-" * 60)
print(f"{'n':>3} | {'Рекурсивно (мс)':>15} | {'Итерационно (мс)':>15} | {'Значение':>15}")
print("-" * 60)

for n in n_values:
    value, iter_time = measure_time(fibonacci_iterative, n)
    iterative_times.append(iter_time)

    _, rec_time = measure_time(fibonacci_recursive, n)
    recursive_times.append(rec_time)

    print(f"{'n':3} | {rec_time:15.6f} | {iter_time:15.6f} | {value:15}")

print("-" * 60)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
```

```

ax1.plot(n_values, iterative_times, 'g-', linewidth=2, label='Итерационный')
ax1.set_xlabel('Номер элемента (n)', fontsize=12)
ax1.set_ylabel('Время выполнения (мс)', fontsize=12)
ax1.set_title('Итерационное вычисление', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)
ax1.legend()

ax2.plot(n_values, recursive_times, 'r-', linewidth=2, label='Рекурсивный')
ax2.set_xlabel('Номер элемента (n)', fontsize=12)
ax2.set_ylabel('Время выполнения (мс)', fontsize=12)
ax2.set_title('Рекурсивное вычисление', fontsize=14, fontweight='bold')
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.suptitle('Сравнение времени вычисления чисел Фибоначчи', fontsize=16,
fontweight='bold', y=1.02)

plt.tight_layout()
plt.savefig('fibonacci_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Пример выполнения:

n	Рекурсивно (мс)	Итерационно (мс)	Значение
1	0.000800	0.001200	1
2	0.001400	0.003400	1
3	0.002700	0.001100	2
4	0.002400	0.002200	3
5	0.002200	0.002200	5
6	0.002900	0.001400	8
7	0.004200	0.001000	13
8	0.005400	0.000900	21
9	0.010500	0.000900	34
10	0.017500	0.003900	55
11	0.025100	0.001500	89
12	0.039700	0.001400	144
13	0.062700	0.001500	233
14	0.108900	0.001600	377
15	0.165300	0.001900	610
16	0.248100	0.002000	987
17	0.540600	0.005800	1597
18	1.032000	0.005200	2584
19	1.170500	0.005400	4181
20	1.639300	0.005800	6765
21	1.965900	0.004600	10946
22	4.556600	0.002500	17711
23	8.981100	0.008800	28657
24	12.681000	0.009700	46368
25	17.134100	0.010500	75025
26	30.882300	0.023200	121393
27	44.252100	0.007500	196418

Рисунок 1 – Пример выполнения первого задания

Графики времени выполнения и количества итераций для методов.

Сравнение времени вычисления чисел Фибоначчи

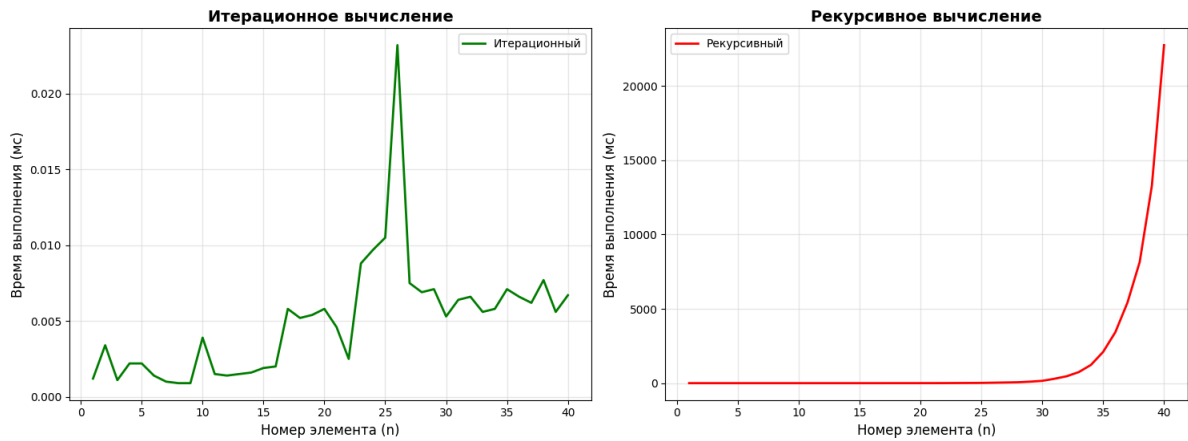


Рисунок 2 – Графики времени выполнения и количества итераций первого задания

Задание 2

Необходимо сравнить три алгоритма сортировки:

- 1) Гномий;
- 2) «Пузырек»;
- 3) Вставками.

Код реализации:

```
import random
import time
import matplotlib.pyplot as plt
import numpy as np

def bubble_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    arr_copy = arr.copy()

    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr_copy[j] > arr_copy[j + 1]:
                arr_copy[j], arr_copy[j + 1] = arr_copy[j + 1], arr_copy[j]
                swaps += 1

    return arr_copy, comparisons, swaps

def gnome_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    arr_copy = arr.copy()

    index = 0
    while index < n:
        if index == 0:
            index += 1
        comparisons += 1
        if arr_copy[index] ≥ arr_copy[index - 1]:
            index += 1
        else:
            arr_copy[index], arr_copy[index - 1] = arr_copy[index - 1],
arr_copy[index]
            swaps += 1
            index -= 1

    return arr_copy, comparisons, swaps

def insertion_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
```

```

arr_copy = arr.copy()

for i in range(1, n):
    key = arr_copy[i]
    j = i - 1
    while j ≥ 0 and arr_copy[j] > key:
        comparisons += 1
        arr_copy[j + 1] = arr_copy[j]
        swaps += 1
        j -= 1
    if j ≥ 0:
        comparisons += 1
    arr_copy[j + 1] = key
    if j + 1 ≠ i:
        swaps += 1

return arr_copy, comparisons, swaps

def measure_sort_time(sort_func, arr):
    start_time = time.perf_counter()
    sorted_arr, comparisons, swaps = sort_func(arr)
    end_time = time.perf_counter()

    execution_time = (end_time - start_time) * 1000
    return execution_time, comparisons, swaps

def generate_random_array(size, min_val=1, max_val=1000):
    return [random.randint(min_val, max_val) for _ in range(size)]

def measure_sort_time(sort_func, arr):
    start_time = time.perf_counter()
    sorted_arr, comparisons, swaps = sort_func(arr)
    end_time = time.perf_counter()

    execution_time = (end_time - start_time) * 1000
    return execution_time, comparisons, swaps

def generate_random_array(size, min_val, max_val):
    return [random.randint(min_val, max_val) for _ in range(size)]

sizes = [10, 50, 100, 200, 500, 1000, 2000, 3000, 4000, 5000]

MIN_VALUE = 1
MAX_VALUE = 10000

bubble_times = []
gnome_times = []
insertion_times = []

bubble_comparisons = []
gnome_comparisons = []
insertion_comparisons = []

bubble_swaps = []
gnome_swaps = []
insertion_swaps = []

print("=" * 100)
print(f"{'Размер':^10} | {'Пузырек (мс)':^15} | {'Гномья (мс)':^15} |  

{'Вставки (мс)':^15} | {'Сравнения (П/Г/В)':^35}")

```

```

print("=" * 100)

for size in sizes:
    arr = generate_random_array(size, MIN_VALUE, MAX_VALUE)

    bubble_time, bubble_comp, bubble_swap = measure_sort_time(bubble_sort,
arr)
    bubble_times.append(bubble_time)
    bubble_comparisons.append(bubble_comp)
    bubble_swaps.append(bubble_swap)

    gnome_time, gnome_comp, gnome_swap = measure_sort_time(gnome_sort, arr)
    gnome_times.append(gnome_time)
    gnome_comparisons.append(gnome_comp)
    gnome_swaps.append(gnome_swap)

    insertion_time, insertion_comp, insertion_swap =
measure_sort_time(insertion_sort, arr)
    insertion_times.append(insertion_time)
    insertion_comparisons.append(insertion_comp)
    insertion_swaps.append(insertion_swap)

    print(f"{size:^10} | {bubble_time:^15.3f} | {gnome_time:^15.3f} |
{insertion_time:^15.3f} | "
          f"{bubble_comp:^10}|{gnome_comp:^10}|{insertion_comp:^10}")

print("=" * 100)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

ax1.plot(sizes, bubble_times, 'ro-', linewidth=2, markersize=6,
label='Пузырек')
ax1.plot(sizes, gnome_times, 'gs-', linewidth=2, markersize=6,
label='Гномья')
ax1.plot(sizes, insertion_times, 'b^-', linewidth=2, markersize=6,
label='Вставки')

ax1.set_xlabel('Размер входных данных (n)', fontsize=12)
ax1.set_ylabel('Время выполнения (мс)', fontsize=12)
ax1.set_title('Сравнение времени выполнения сортировок', fontsize=14,
fontweight='bold')
ax1.grid(True, alpha=0.3, linestyle='--')
ax1.legend(fontsize=11)

ax2.plot(sizes, bubble_comparisons, 'ro-', linewidth=2, markersize=6,
label='Пузырек')
ax2.plot(sizes, gnome_comparisons, 'gs-', linewidth=2, markersize=6,
label='Гномья')
ax2.plot(sizes, insertion_comparisons, 'b^-', linewidth=2, markersize=6,
label='Вставки')

ax2.set_xlabel('Размер входных данных (n)', fontsize=12)
ax2.set_ylabel('Количество сравнений', fontsize=12)
ax2.set_title('Показатель качества сортировки (сравнения)', fontsize=14,
fontweight='bold')
ax2.grid(True, alpha=0.3, linestyle='--')
ax2.legend(fontsize=11)

```

```
plt.suptitle('Сравнительный анализ алгоритмов сортировки', fontsize=16,
fontweight='bold', y=1.02)

plt.tight_layout()
plt.savefig('sorting_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"{'Алгоритм':<15} | {'Среднее время (мс)':<20} | {'Среднее сравнений':<20} | {'Среднее обменов':<20}")
print("-" * 100)
print(f"{'Пузырек':<15} | {np.mean(bubble_times):<20.3f} | {np.mean(bubble_comparisons):<20.0f} | {np.mean(bubble_swaps):<20.0f}")
print(f"{'Гномья':<15} | {np.mean(gnome_times):<20.3f} | {np.mean(gnome_comparisons):<20.0f} | {np.mean(gnome_swaps):<20.0f}")
print(f"{'Вставки':<15} | {np.mean(insertion_times):<20.3f} | {np.mean(insertion_comparisons):<20.0f} | {np.mean(insertion_swaps):<20.0f}")
```

Пример выполнения:

Размер	Пузырек (мс)	Гномья (мс)	Вставки (мс)	Сравнения (П/Г/В)		
10	0.009	0.005	0.004	45	40	24
50	0.082	0.097	0.044	1225	1380	713
100	0.293	0.459	0.182	4950	5014	2554
200	1.222	4.612	0.661	19900	20136	10164
500	8.870	11.149	4.582	124750	129459	64975
1000	38.984	40.550	18.202	499500	503200	252097
2000	147.544	169.679	69.782	1999000	1984071	993030
3000	331.611	370.528	166.417	4498500	4404874	2203932
4000	611.456	671.375	281.141	7998000	7991318	3997654
5000	901.497	1020.975	439.455	12497500	12462034	6233513

Рисунок 3 – Пример выполнения второго задания

Графики времени выполнения и количества сравнений для алгоритмов.

Сравнительный анализ алгоритмов сортировки

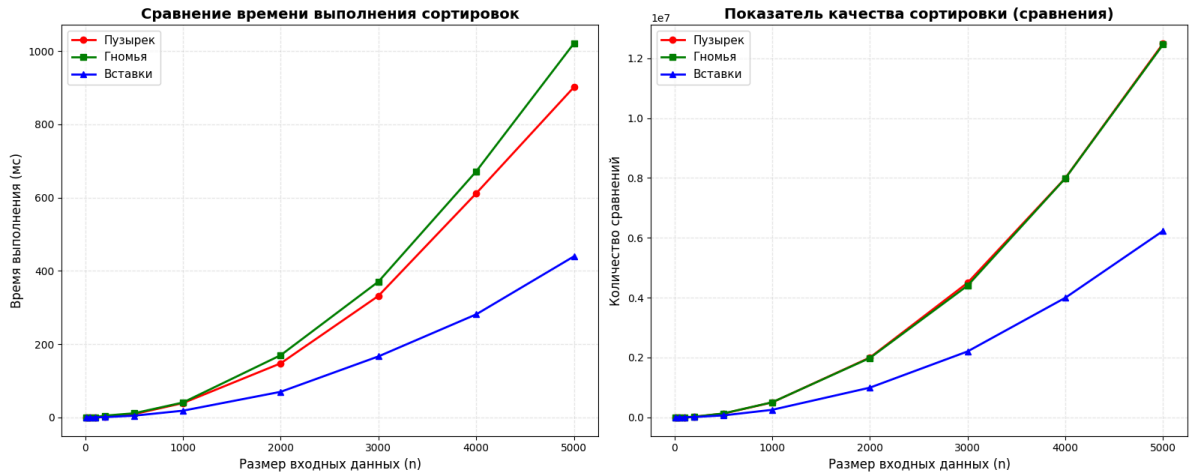


Рисунок 4 – Графики времени выполнения и количества сравнений для алгоритмов второго задания

Задание 3

Необходимо реализовать 3 алгоритма генерации всех перестановок:

- 1) Нарайаны;
- 2) Джонсона-Троттера;
- 3) Вектора инверсий.

Реализовать алгоритм генерации уникальных перестановок для последовательностей с повторениями, перебор всех возможных выборов из n элементов, решить прикладную задачу оптимизации жадным методом.

Код реализации:

```
import time
import itertools
import numpy as np
from math import factorial

def measure_time(func, *args):
    start = time.perf_counter()
    result = func(*args)
    elapsed = (time.perf_counter() - start) * 1000
    return result, elapsed

def narayana_permutations(n):
    if n == 0:
        return []
    perm = list(range(1, n + 1))
    result = [perm[:]]
    while True:
        i = n - 2
        while i >= 0 and perm[i] >= perm[i + 1]:
            i -= 1
        if i < 0:
            break
        j = n - 1
        while perm[j] <= perm[i]:
            j -= 1
        perm[i], perm[j] = perm[j], perm[i]
        perm[i + 1:] = perm[i + 1:][::-1]
        result.append(perm[:])
    return result

def johnson_trotter_permutations(n):
    if n == 0:
        return []
    elements = list(range(1, n + 1))
    directions = [-1] * n
```

```

result = [elements[:]]

def is_mobile(i):
    d = directions[i]
    ni = i + d
    return 0 ≤ ni < n and elements[i] > elements[ni]

while True:
    mobile_val = -1
    mobile_idx = -1
    for i in range(n):
        if is_mobile(i) and elements[i] > mobile_val:
            mobile_val = elements[i]
            mobile_idx = i
    if mobile_idx == -1:
        break

    ni = mobile_idx + directions[mobile_idx]
    elements[mobile_idx], elements[ni] = elements[ni],
elements[mobile_idx]
    directions[mobile_idx], directions[ni] = directions[ni],
directions[mobile_idx]
    mobile_idx = ni

    for i in range(n):
        if elements[i] > mobile_val:
            directions[i] = -directions[i]
    result.append(elements[:])
return result

def inversion_vector_permutations(n):
    if n == 0:
        return []
    result = []
    total = factorial(n)
    for k in range(total):
        inv = []
        num = k
        for i in range(1, n + 1):
            inv.append(num % i)
            num //= i
        inv.reverse()
        available = list(range(1, n + 1))
        perm = []
        for idx in inv:
            perm.append(available.pop(idx))
        result.append(perm)
    return result

def unique_permutations(sequence):
    elements = sorted(sequence)
    result = []

    def backtrack(current, remaining):
        if not remaining:
            result.append(current[:])
            return
        seen = set()

```

```

        for i in range(len(remaining)):
            if remaining[i] not in seen:
                seen.add(remaining[i])
                current.append(remaining[i])
                backtrack(current, remaining[:i] + remaining[i + 1:])
                current.pop()

    backtrack([], elements)
    return result

def all_subsets(elements):
    result = []
    n = len(elements)
    for r in range(1, n + 1):
        for combo in itertools.combinations(elements, r):
            result.append(list(combo))
    return result

def stationery_purchase(budget, shopping_list, price_list):
    available = [
        (name, qty, price_list[name])
        for name, qty in shopping_list
        if name in price_list
    ]
    available.sort(key=lambda x: x[2])

    selected = {}
    remaining_budget = budget

    for name, qty_needed, price in available:
        if price ≤ remaining_budget:
            selected[name] = 1
            remaining_budget -= price

    for name, qty_needed, price in available:
        if name in selected:
            while selected[name] < qty_needed and price ≤ remaining_budget:
                selected[name] += 1
                remaining_budget -= price

    spent = budget - remaining_budget
    return selected, spent

print("=" * 75)
print("ЗАДАНИЕ 1: Сравнение алгоритмов генерации перестановок (n = 1..10)")
print("=" * 75)

test_ns = list(range(1, 11))
t_narayana = []
t_johnson = []
t_inversion = []

header = (f"{'n':^5} | {'n!':^10} | "
          f"{'Нарайана (мс)':^16} | {'Джонсон-Троттер (мс)':^22} | "
          f"{'Вектор инверсий (мс)':^22}")
print(header)
print("-" * len(header))

```

```

for n in test_ns:
    _, t1 = measure_time(narayana_permutations, n)
    _, t2 = measure_time(johnson_trotter_permutations, n)
    _, t3 = measure_time(inversion_vector_permutations, n)
    t_narayana.append(t1)
    t_johnson.append(t2)
    t_inversion.append(t3)
    print(f"{n:^5} | {factorial(n):^10} | {t1:^16.4f} | {t2:^22.4f} | {t3:^22.4f}")

print()
print(f"Средн. время (мс): Нарайана={np.mean(t_narayana):.4f}  "
      f"Джонсон-Троттер={np.mean(t_johnson):.4f}  "
      f"Вектор инверсий={np.mean(t_inversion):.4f}")

print()
print("=" * 75)
print("ЗАДАНИЕ 2: Уникальные перестановки (с повторяющимися элементами)")
print("=" * 75)

task2_cases = [
    [1, 2, 1],
    [1, 2, 3],
    [1, 1, 1],
    [1, 2, 2, 3],
]

for seq in task2_cases:
    perms, t = measure_time(unique_permutations, seq)
    print(f"\nПоследовательность: {''.join(map(str, seq))}  "
          f"→ {len(perms)} перестановок (время: {t:.4f} мс)")
    for p in perms:
        print(f"    {''.join(map(str, p))}")

print()
print("Время для последовательностей разной длины:")
print(f"{'Длина':^8} | {'Кол-во перест.':^16} | {'Время (мс)':^14}")
print("-" * 45)
for length in range(1, 9):
    seq = list(range(1, length + 1))
    perms, t = measure_time(unique_permutations, seq)
    print(f"{length:^8} | {len(perms):^16} | {t:^14.4f}")

print()
print("=" * 75)
print("ЗАДАНИЕ 3: Все возможные выборки из n элементов")
print("=" * 75)

items = ["стол", "стул", "шкаф"]
subsets, t = measure_time(all_subsets, items)
print(f"\nЭлементы: {items}")
print(f"Всего вариантов выборки: {len(subsets)} (время: {t:.4f} мс)")
for s in subsets:
    print(f"    {s}")

print()
print("Время для разного числа элементов:")
print(f"{'n':^6} | {'Кол-во выборок':^16} | {'Время (мс)':^14}")
print("-" * 42)
t_subsets = []

```

```

subset_ns = list(range(1, 16))
for n in subset_ns:
    elems = list(range(1, n + 1))
    subs, t = measure_time(all_subsets, elems)
    t_subsets.append(t)
    print(f"{n:^6} | {len(subs):^16} | {t:^14.4f}")

print()
print("=" * 75)
print("ЗАДАНИЕ 4: Покупка канцелярских принадлежностей студента Петрова П.П.")
print("=" * 75)

budget = 500.0
shopping_list = [
    ("Ручка", 3),
    ("Тетрадь", 2),
    ("Карандаш", 5),
    ("Линейка", 1),
    ("Ластик", 2),
    ("Скрепки", 1),
    ("Папка", 1),
]
price_list = {
    "Ручка": 30.0,
    "Тетрадь": 80.0,
    "Карандаш": 15.0,
    "Линейка": 50.0,
    "Ластик": 20.0,
    "Скрепки": 25.0,
    "Папка": 120.0,
    "Степлер": 200.0,
}

print(f"\nБюджет студента: {budget:.2f} руб.")
print("\nСписок необходимых покупок:")
print(f" {'Наименование':<15} | {'Нужно, шт.':^12} | {'Цена/шт., руб.':^16}")
print(" " + "-" * 48)
for name, qty in shopping_list:
    price = price_list.get(name, "-")
    price_str = f"{price:.2f}" if isinstance(price, float) else price
    print(f" {name:<15} | {qty:^12} | {price_str:^16}")

(selected, spent), t_task4 = measure_time(stationery_purchase, budget,
shopping_list, price_list)

print(f"\nРезультат (время поиска: {t_task4:.4f} мс):")
print(f" {'Наименование':<15} | {'Куплено, шт.':^14} | {'Сумма, руб.':^14}")
print(" " + "-" * 48)
for name, qty in selected.items():
    total_price = qty * price_list[name]
    print(f" {name:<15} | {qty:^14} | {total_price:^14.2f}")
print(" " + "-" * 48)
print(f" Куплено наименований: {len(selected)} из {len(shopping_list)}")
print(f" Потрачено: {spent:.2f} руб. | Остаток: {budget - spent:.2f} руб.")

```

Пример выполнения:

=====								
ЗАДАНИЕ 1: Сравнение алгоритмов генерации перестановок (n = 1..10)								
=====								
n		n!		Нарайана (мс)		Джонсон-Троттер (мс)		Вектор инверсий (мс)

1		1		0.0041		0.0051		0.0076
2		2		0.0047		0.0088		0.0089
3		6		0.0113		0.0181		0.0102
4		24		0.0201		0.0381		0.0281
5		120		0.0750		0.1991		0.1580
6		720		1.2153		1.3352		1.1132
7		5040		3.3384		10.6703		8.6820
8		40320		78.1666		100.4209		105.8404
9		362880		623.2790		1487.1280		1028.1939
10		3628800		5390.7680		14323.6212		11025.0977
Средн. время (мс): Нарайана=609.6883 Джонсон-Троттер=1592.3445 Вектор инверсий=1216.9140								

Рисунок 5 – Пример выполнения первой части третьего задания

ЗАДАНИЕ 2: Уникальные перестановки (с повторяющимися элементами)

=====

Последовательность: 121 -> 3 перестановок (время: 0.0282 мс)

112
121
211

Последовательность: 123 -> 6 перестановок (время: 0.0157 мс)

123
132
213
231
312
321

Последовательность: 111 -> 1 перестановок (время: 0.0047 мс)

111

Последовательность: 1223 -> 12 перестановок (время: 0.0257 мс)

1223
1232
1322
2123
2132
2213
2231
2312
2321
3122

Рисунок 6 – Пример выполнения второй части третьего задания

ЗАДАНИЕ 3: Все возможные выборки из n элементов

Элементы: ['стол', 'стул', 'шкаф']

Всего вариантов выборки: 7 (время: 0.0131 мс)

```
['стол']  
['стул']  
['шкаф']  
['стол', 'стул']  
['стол', 'шкаф']  
['стул', 'шкаф']  
['стол', 'стул', 'шкаф']
```

Время для разного числа элементов:

n	Кол-во выборок	Время (мс)
1	1	0.0033
2	3	0.0030
3	7	0.0035
4	15	0.0049
5	31	0.0080
6	63	0.0140
7	127	0.0242
8	255	0.0520
9	511	0.1652
10	1023	0.2184
11	2047	0.3733
12	4095	2.8828
13	8191	1.7281
14	16383	2.6989

Рисунок 7 – Пример выполнения третьей части третьего задания

=====

ЗАДАНИЕ 4: Покупка канцелярских принадлежностей студента Петрова П.П.

=====

Бюджет студента: 500.00 руб.

Список необходимых покупок:

Наименование	Нужно, шт.	Цена/шт., руб.
-----	-----	-----
Ручка	3	30.00
Тетрадь	2	80.00
Карандаш	5	15.00
Линейка	1	50.00
Ластик	2	20.00
Скрепки	1	25.00
Папка	1	120.00

Результат (время поиска: 0.0178 мс):

Наименование	Куплено, шт.	Сумма, руб.
-----	-----	-----
Карандаш	5	75.00
Ластик	2	40.00
Скрепки	1	25.00
Ручка	3	90.00
Линейка	1	50.00
Тетрадь	1	80.00
Папка	1	120.00
-----	-----	-----

Куплено наименований: 7 из 7

Потрачено: 480.00 руб. | Остаток: 20.00 руб.

Рисунок 8 – Пример выполнения четвёртой части третьего задания

Задание 4

Необходимо сгенерировать перестановки значений $1, \dots, n$, используя код Грэя и выполнить сравнительный анализ алгоритмов генерации случайных перестановок.

Код реализации:

```
import time
import matplotlib.pyplot as plt
import numpy as np
import random

def johnson_trotter_permutations(n):
    elements = list(range(1, n + 1))
    directions = [-1] * n
    permutations = [elements[:]]

    def get_mobile_idx():
        mobile_idx = -1
        for i in range(n):
            next_idx = i + directions[i]
            if 0 ≤ next_idx < n:
                if elements[i] > elements[next_idx]:
                    if mobile_idx == -1 or elements[i] >
elements[mobile_idx]:
                        mobile_idx = i
        return mobile_idx

    while True:
        m_idx = get_mobile_idx()
        if m_idx == -1:
            break

        mobile_val = elements[m_idx]
        target_idx = m_idx + directions[m_idx]

        elements[m_idx], elements[target_idx] = elements[target_idx],
elements[m_idx]
        directions[m_idx], directions[target_idx] = directions[target_idx],
directions[m_idx]

        for i in range(n):
            if elements[i] > mobile_val:
                directions[i] *= -1

        permutations.append(elements[:])

    return permutations

n = 3
start = time.perf_counter()
res = johnson_trotter_permutations(n)
elapsed = (time.perf_counter() - start) * 1000
```

```

print(f"Перестановки для n={n}:")
for p in res:
    print(p)

print(f"\nВсего перестановок: {len(res)}")
print(f"Время выполнения: {elapsed:.4f} мс")

def naive_shuffle(n):
    arr = list(range(1, n + 1))
    weights = [random.random() for _ in range(n)]
    combined = list(zip(weights, arr))
    combined.sort()
    return [x[1] for x in combined]

def fisher_yates_shuffle(n):
    arr = list(range(1, n + 1))
    for i in range(n - 1, 0, -1):
        j = random.randint(0, i)
        arr[i], arr[j] = arr[j], arr[i]
    return arr

def builtin_shuffle(n):
    arr = list(range(1, n + 1))
    random.shuffle(arr)
    return arr

sizes = [10, 100, 500, 1000, 2000, 5000]
results = {"Naive": [], "Fisher-Yates": [], "Builtin": []}

for n in sizes:
    start = time.perf_counter()
    naive_shuffle(n)
    results["Naive"].append((time.perf_counter() - start) * 1000)

    start = time.perf_counter()
    fisher_yates_shuffle(n)
    results["Fisher-Yates"].append((time.perf_counter() - start) * 1000)

    start = time.perf_counter()
    builtin_shuffle(n)
    results["Builtin"].append((time.perf_counter() - start) * 1000)

plt.figure(figsize=(10, 6))
plt.plot(sizes, results["Naive"], label='Наивный (Sort  $O(n \log n)$ )',
marker='o')
plt.plot(sizes, results["Fisher-Yates"], label='Фишер-Йейтс ( $O(n)$ )',
marker='s')
plt.plot(sizes, results["Builtin"], label='Встроенный random.shuffle',
marker='^')

plt.title('Сравнение алгоритмов генерации случайных перестановок')
plt.xlabel('Размер массива (n)')
plt.ylabel('Время выполнения (мс)')
plt.legend()
plt.grid(True)
plt.show()

```

Пример выполнения:

```
Перестановки для n=3:  
[1, 2, 3]  
[1, 3, 2]  
[3, 1, 2]  
[3, 2, 1]  
[2, 3, 1]  
[2, 1, 3]  
  
Всего перестановок: 6  
Время выполнения: 0.2065 мс
```

Рисунок 9 – Пример выполнения четвертого задания

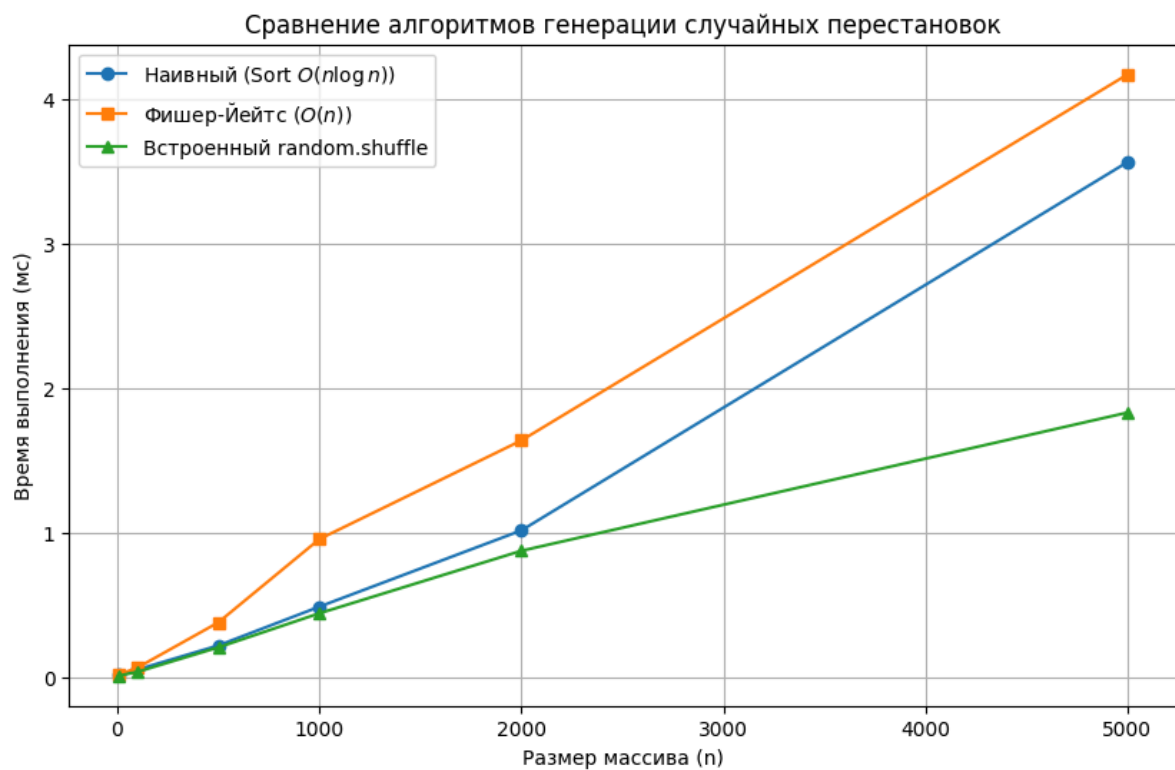


Рисунок 10 – Сравнительный анализ алгоритмов генерации случайных перестановок

Задание 5

Выполнить сравнительный анализ работы алгоритмов по поиску подстроки в строке (Наивный метод, Алгоритм Бойера-Мура, Алгоритм Рабина-Карпа, Алгоритм Кнута-Морриса-Пратта)

Код реализации:

```
import time
import matplotlib.pyplot as plt

def naive_search(text, pattern):
    n, m = len(text), len(pattern)
    res = []
    for i in range(n - m + 1):
        if text[i:i+m] == pattern:
            res.append(i)
    return res

def kmp_search(text, pattern):
    def compute_prefix(p):
        m = len(p)
        pi = [0] * m
        k = 0
        for q in range(1, m):
            while k > 0 and p[k] != p[q]:
                k = pi[k-1]
            if p[k] == p[q]:
                k += 1
            pi[q] = k
        return pi

    n, m = len(text), len(pattern)
    pi = compute_prefix(pattern)
    q = 0
    res = []
    for i in range(n):
        while q > 0 and pattern[q] != text[i]:
            q = pi[q-1]
        if pattern[q] == text[i]:
            q += 1
        if q == m:
            res.append(i - m + 1)
            q = pi[q-1]
    return res

def rabin_karp_search(text, pattern, d=256, q=101):
    n, m = len(text), len(pattern)
    h = pow(d, m-1) % q
    p = 0
    t = 0
    res = []
    for i in range(m):
        p = (d * p + ord(pattern[i])) % q
        t = (d * t + ord(text[i])) % q
```

```

for s in range(n - m + 1):
    if p == t:
        if text[s:s+m] == pattern:
            res.append(s)
    if s < n - m:
        t = (d * (t - ord(text[s]) * h) + ord(text[s+m])) % q
        t = (t + q) % q
return res

def boyer_moore_search(text, pattern):
    m = len(pattern)
    n = len(text)
    if m == 0: return []

    bad_char = {pattern[i]: i for i in range(m)}

    res = []
    s = 0
    while s ≤ n - m:
        j = m - 1
        while j ≥ 0 and pattern[j] == text[s + j]:
            j -= 1
        if j < 0:
            res.append(s)
            s += (m - bad_char.get(text[s + m], -1)) if s + m < n else 1
        else:
            s += max(1, j - bad_char.get(text[s + j], -1))
    return res

def run_benchmarks():
    big_text = "ABC" * 30000 + "SEARCH_ME" + "ABC" * 10000
    pattern = "SEARCH_ME"

    methods = [
        ("Naive", naive_search),
        ("KMP", kmp_search),
        ("Rabin-Karp", rabin_karp_search),
        ("Boyer-Moore", boyer_moore_search)
    ]

    results = {}

    print(f"{'Алгоритм':<15} | {'Время (мс)':<10}")
    print("-" * 30)

    for name, func in methods:
        start = time.perf_counter()
        func(big_text, pattern)
        end = time.perf_counter()
        elapsed = (end - start) * 1000
        results[name] = elapsed
        print(f"{name:<15} | {elapsed:.4f}")

    plt.bar(results.keys(), results.values(), color=['blue', 'green',
'orange', 'red'])
    plt.ylabel('Время выполнения (мс)')
    plt.title('Сравнение алгоритмов поиска подстроки')
    plt.show()

run_benchmarks()

```

Пример выполнения:

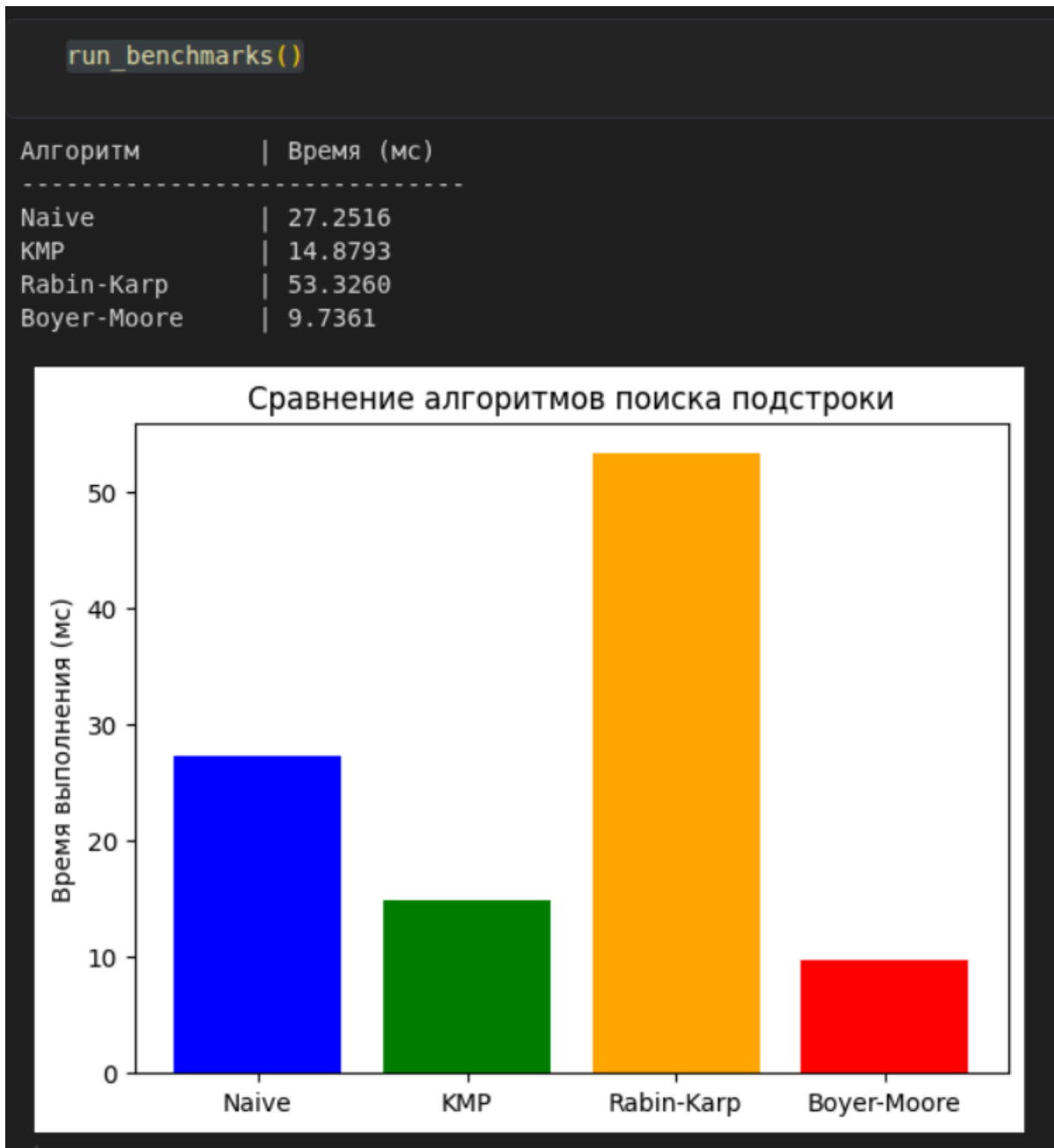


Рисунок 11 – Сравнение алгоритмов поиска подстроки

Задание 6

Необходимо выполнить 3 задачи:

1. На прямой даны n отрезков, нужно выбрать максимальное по размеру подмножество непересекающихся.
2. Для конкретной аудитории есть ряд заявок с временем начала и окончания работы. Необходимо вернуть максимальное количество допустимых заявок.
3. На прямой расположены отрезки $[li, ri]$, которые полностью или даже с избытком покрывают интервал $[L, R]$. необходимо выбрать наименьшее множество отрезков так, чтобы они всё ещё покрывали интервал.

Код реализации:

```
def max_non_overlapping_intervals(intervals):
    intervals.sort(key=lambda x: x[1])

    result = []
    if not intervals:
        return result

    last_r = intervals[0][1]
    result.append(intervals[0])

    for i in range(1, len(intervals)):
        if intervals[i][0] > last_r:
            result.append(intervals[i])
            last_r = intervals[i][1]

    return result

data = [(1, 4), (4, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11),
        (2, 13), (12, 14)]
print("Максимальное подмножество:", max_non_overlapping_intervals(data))

def max_applications(apps):
    apps.sort(key=lambda x: x[1])

    selected = []
    last_end_time = -1

    for start, end in apps:
        if start >= last_end_time:
            selected.append((start, end))
            last_end_time = end

    return selected
```

```

applications = [(9, 11), (10, 12), (11, 13), (12, 14), (13, 15)]
print("Выбранные заявки:", max_applications(applications))

def min_interval_cover(intervals, target_L, target_R):
    intervals.sort()

    selected = []
    current_right = target_L
    i = 0
    n = len(intervals)

    while current_right < target_R:
        best_r = current_right
        best_interval = None

        while i < n and intervals[i][0] ≤ current_right:
            if intervals[i][1] > best_r:
                best_r = intervals[i][1]
                best_interval = intervals[i]
            i += 1

        if best_interval is None:
            return "Покрытие невозможно"

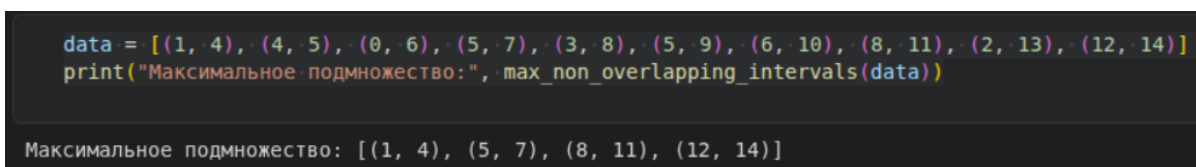
        selected.append(best_interval)
        current_right = best_r

    return selected

test_intervals = [(0, 4), (2, 7), (5, 10), (8, 12)]
print("Минимальное покрытие [0, 10]:", min_interval_cover(test_intervals, 0,
10))

```

Пример выполнения:



```

data = [(1, 4), (4, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (2, 13), (12, 14)]
print("Максимальное подмножество:", max_non_overlapping_intervals(data))

```

Максимальное подмножество: [(1, 4), (5, 7), (8, 11), (12, 14)]

Рисунок 12 – Пример выполнения первой части шестого задания

```
applications = [(9, 11), (10, 12), (11, 13), (12, 14), (13, 15)]  
print("Выбранные заявки:", max_applications(applications))
```

Выбранные заявки: [(9, 11), (11, 13), (13, 15)]

Рисунок 13 – Пример выполнения второй части шестого задания

```
test_intervals = [(0, 4), (2, 7), (5, 10), (8, 12)]  
print("Минимальное покрытие [0, 10]:", min_interval_cover(test_intervals, 0, 10))
```

Минимальное покрытие [0, 10]: [(0, 4), (2, 7), (5, 10)]

Рисунок 14 – Пример выполнения третьей части шестого задания

Задание 7

Необходимо выполнить 2 задачи:

1. Имеется m дней и n заказов, каждый из которых займет один день. У каждого заказа есть номер, стоимость и дедлайн. Каждый заказ можно начинать в любой день до дедлайна, одновременно можно делать только один заказ. Определить наилучшее по суммарной стоимости число заказов.

2. Необходимо организовать детский утренник на который пришло n детей. У каждого ребёнка есть номер и возраст. Определить минимальное количество групп, которые будут организованы, но при этом возраст детей в каждой группе не должен отличаться больше, чем на 2 года.

Код реализации:

```
def solve_task_1(jobs):

    max_deadline = max(job[1] for job in jobs)
    schedule = [None] * (max_deadline + 1)

    jobs.sort(key=lambda x: x[2], reverse=True)

    total_profit = 0
    selected_jobs = []

    for job in jobs:
        name, deadline, price = job
        for day in range(deadline, 0, -1):
            if schedule[day] is None:
                schedule[day] = name
                total_profit += price
                selected_jobs.append(name)
                break

    return selected_jobs, total_profit
```

```

data1 = [('A', 2, 40), ('B', 1, 25), ('C', 2, 30), ('D', 1, 15), ('E', 3, 20)]
jobs_res, profit = solve_task_1(data1)
print(f"Выбранные заказы: {jobs_res}, Общая стоимость: {profit}")

def solve_task_2(children):
    if not children:
        return 0, []

    children.sort()

    groups = []
    if children:
        current_group = [children[0]]
        for i in range(1, len(children)):
            if children[i] - current_group[0] ≤ 2:
                current_group.append(children[i])
            else:
                groups.append(current_group)
                current_group = [children[i]]
        groups.append(current_group)

    return len(groups), groups

kids_ages = [5, 3, 8, 4, 10, 2, 7]
count, distribution = solve_task_2(kids_ages)
print(f"Количество групп: {count}, Распределение: {distribution}")

```

Пример выполнения:

```

data1 = [('A', 2, 40), ('B', 1, 25), ('C', 2, 30), ('D', 1, 15), ('E', 3, 20)]
jobs_res, profit = solve_task_1(data1)
print(f"Выбранные заказы: {jobs_res}, Общая стоимость: {profit}")

```

```

Выбранные заказы: ['A', 'C', 'E'], Общая стоимость: 90

```

Рисунок 15 – Пример выполнения первой части седьмого задания

```
kids_ages = [5, 3, 8, 4, 10, 2, 7]
count, distribution = solve_task_2(kids_ages)
print(f"Количество групп: {count}, Распределение: {distribution}")
```

Количество групп: 3, Распределение: [[2, 3, 4], [5, 7], [8, 10]]

Рисунок 16 – Пример выполнения второй части седьмого задания

Задание 8

Необходимо выполнить задачу:

Дана последовательность элементов с заданным весом и стоимостью, каждый элемент может встречаться только один раз. Необходимо заполнить рюкзак предметами, суммарная стоимость которых была бы как можно большей, грузоподъемность рюкзака ограничивается W килограммами, где W – целая величина. Определить предметы, которые будут помещены в рюкзак. Решение выполнить с использованием полного перебора и жадным алгоритмом. Оценить сложность работы алгоритмов.

Код реализации:

```
import itertools
import time

class Item:
    def __init__(self, name, weight, value):
        self.name = name
        self.weight = weight
        self.value = value
        self.ratio = value / weight

def brute_force_knapsack(items, capacity):
    best_value = -1
    best_combination = []

    n = len(items)
    for i in range(n + 1):
        for combination in itertools.combinations(items, i):
            total_weight = sum(item.weight for item in combination)
            total_value = sum(item.value for item in combination)

            if total_weight ≤ capacity and total_value > best_value:
                best_value = total_value
                best_combination = combination
```

```

    return best_combination, best_value

def greedy_knapsack(items, capacity):
    sorted_items = sorted(items, key=lambda x: x.ratio, reverse=True)

    total_value = 0
    total_weight = 0
    knapsack = []

    for item in sorted_items:
        if total_weight + item.weight ≤ capacity:
            knapsack.append(item)
            total_weight += item.weight
            total_value += item.value

    return knapsack, total_value

items_list = [
    Item("Золотой слиток", 10, 60),
    Item("Картина", 20, 100),
    Item("Ваза", 30, 120),
    Item("Часы", 5, 50),
    Item("Ноутбук", 15, 90)
]
W = 35

print(f"Грузоподъемность рюкзака: {W}\n")

start = time.time()
bf_items, bf_val = brute_force_knapsack(items_list, W)
end = time.time()
print("Полный перебор")
print(f"Предметы: {[item.name for item in bf_items]}")
print(f"Максимальная стоимость: {bf_val}")
print(f"Время выполнения: {end - start:.6f} сек.\n")

start = time.time()
gr_items, gr_val = greedy_knapsack(items_list, W)
end = time.time()

```

```
print("Жадный алгоритм")
print(f"Предметы: {[item.name for item in gr_items]}")
print(f"Суммарная стоимость: {gr_val}")
print(f"Время выполнения: {end - start:.6f} сек.")
```

Пример выполнения:

```
... Грузоподъемность рюкзака: 35

Полный перебор
Предметы: ['Золотой слиток', 'Картина', 'Часы']
Максимальная стоимость: 210
Время выполнения: 0.000170 сек.

Жадный алгоритм
Предметы: ['Часы', 'Золотой слиток', 'Ноутбук']
Суммарная стоимость: 200
Время выполнения: 0.000132 сек.
```

Рисунок 17 – Пример выполнения восьмого задания

Задание 9

Необходимо выполнить 2 задачи:

3. Задача о распределении работ между механизмами. Заданы множества $V = \{v_1, v_2, \dots, v_n\}$ и $S = \{s_1, s_2, \dots, s_m\}$ работ и механизмов соответственно. Для выполнения каждой из работ требуется некоторое время, одинаковое для всех работ, и некоторые механизмы. При этом никакой из механизмов не может быть одновременно занят в нескольких работах. Нужно распределить механизмы так, чтобы общее время выполнения всех работ было минимальным. Использовать жадный алгоритм. Оценить сложность работы алгоритма.

4. Задача о размещении грузов. Пусть имеется n грузов, которые необходимо разместить по контейнерам для перевозки, причем некоторые грузы нельзя помещать в один контейнер. Определить наименьшее количество контейнеров, которые потребуются для перевозки всех грузов, определить компоновку грузов по контейнерам. Оценить сложность работы алгоритма.

Код реализации:

```
def greedy_graph_coloring(jobs, mechanisms_map):
    adj = {job: set() for job in jobs}
    for i in range(len(jobs)):
        for j in range(i + 1, len(jobs)):
            job1, job2 = jobs[i], jobs[j]
            if mechanisms_map[job1] & mechanisms_map[job2]:
                adj[job1].add(job2)
                adj[job2].add(job1)

    sorted_jobs = sorted(jobs, key=lambda x: len(adj[x]), reverse=True)

    job_colors = {}
    current_color = 1

    while len(job_colors) < len(jobs):
```

```

        for job in sorted_jobs:
            if job not in job_colors:
                can_color = True
                for neighbor in adj[job]:
                    if job_colors.get(neighbor) == current_color:
                        can_color = False
                        break

                if can_color:
                    job_colors[job] = current_color

            current_color += 1

    return job_colors, current_color - 1

jobs_list = [f"a{i}" for i in range(1, 11)]
data = {
    "a1": {"b1", "b5", "b8"},
    "a2": {"b2", "b3"},
    "a3": {"b2", "b7", "b9"},
    "a4": {"b1", "b6", "b7", "b9"},
    "a5": {"b3", "b5"},
    "a6": {"b1", "b4", "b7", "b9"},
    "a7": {"b2", "b9"},
    "a8": {"b3", "b6", "b8"},
    "a9": {"b2", "b4", "b5"},
    "a10": {"b1", "b6", "b9"}
}

result, total_time = greedy_graph_coloring(list(data.keys()), data)

print(f"Минимальное время выполнения: {total_time} ед.")
for time_slot in range(1, total_time + 1):
    current_jobs = [job for job, slot in result.items() if slot == time_slot]
    print(f"Таймслот {time_slot}: {' '.join(current_jobs)}")

def greedy_cargo_placement(cargos, conflicts):
    adj = {cargo: set(conflicts.get(cargo, set())) for cargo in cargos}
    for cargo in cargos:
        for neighbor in list(adj[cargo]):

```

```

        adj[neighbor].add(cargo)

sorted_cargos = sorted(cargos, key=lambda x: len(adj[x]), reverse=True)

assignment = {}
current_container = 1

while len(assignment) < len(cargos):
    for cargo in sorted_cargos:
        if cargo not in assignment:
            can_place = True
            for neighbor in adj[cargo]:
                if assignment.get(neighbor) == current_container:
                    can_place = False
                    break
            if can_place:
                assignment[cargo] = current_container
            current_container += 1

num_containers = current_container - 1
return assignment, num_containers

cargos_list = ['g1', 'g2', 'g3', 'g4', 'g5', 'g6', 'g7', 'g8', 'g9', 'g10']

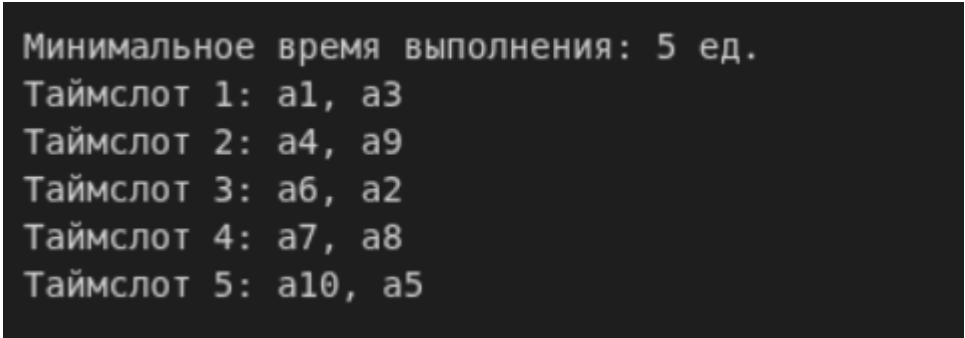
conflicts_data = {
    'g1': {'g2', 'g5', 'g8'},
    'g2': {'g1', 'g3', 'g6'},
    'g3': {'g2', 'g4', 'g7', 'g9'},
    'g4': {'g3', 'g5', 'g8'},
    'g5': {'g1', 'g4', 'g6', 'g10'},
    'g6': {'g2', 'g5', 'g7'},
    'g7': {'g3', 'g6', 'g9'},
    'g8': {'g1', 'g4'},
    'g9': {'g3', 'g7', 'g10'},
    'g10': {'g5', 'g9'}
}

assignment, num_containers = greedy_cargo_placement(cargos_list, conflicts_data)

```

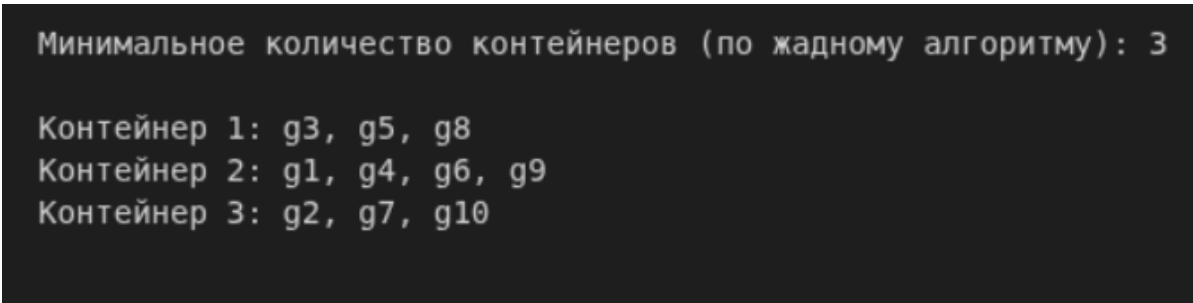
```
print(f"Минимальное количество контейнеров (по жадному алгоритму):  
{num_containers}\n")  
for container in range(1, num_containers + 1):  
    items = [cargo for cargo, cont in assignment.items() if cont == container]  
    print(f"Контейнер {container}: {' '.join(items)}")
```

Пример выполнения:



```
Минимальное время выполнения: 5 ед.  
Таймслот 1: a1, a3  
Таймслот 2: a4, a9  
Таймслот 3: a6, a2  
Таймслот 4: a7, a8  
Таймслот 5: a10, a5
```

Рисунок 18 – Пример выполнения первой части девятого задания



```
Минимальное количество контейнеров (по жадному алгоритму): 3  
  
Контейнер 1: g3, g5, g8  
Контейнер 2: g1, g4, g6, g9  
Контейнер 3: g2, g7, g10
```

Рисунок 19 – Пример выполнения второй части девятого задания

Задание 10

Необходимо выполнить следующую задачу:

На вход программы подается текст, выполнить кодировку текста. Выдать словарь, в котором каждому символу соответствует код, удовлетворяющий условию Фано. Текст должен быть закодирован минимально возможной последовательностью нулей и единиц. Выполнить восстановление закодированного текста. Оценить сложность работы алгоритма.

Код реализации:

```
import heapq
import time
from collections import Counter

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def count_frequencies(text: str) → dict:
    return dict(Counter(text))

def build_huffman_tree(freq: dict) → Node:
    heap = [Node(char, f) for char, f in freq.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
```

```

        parent = Node(None, left.freq + right.freq)
        parent.left = left
        parent.right = right

        heapq.heappush(heap, parent)

    return heap[0]

def build_codes(node: Node, prefix: str = "", codes: dict = None) → dict:
    if codes is None:
        codes = {}

    if node is None:
        return codes

    if node.char is not None:
        codes[node.char] = prefix if prefix else "0"
        return codes

    build_codes(node.left, prefix + "0", codes)
    build_codes(node.right, prefix + "1", codes)
    return codes

def encode(text: str, codes: dict) → str:
    return "".join(codes[ch] for ch in text)

def decode(encoded: str, root: Node) → str:
    result = []
    current = root
    for bit in encoded:
        current = current.left if bit == "0" else current.right
        if current.char is not None:
            result.append(current.char)
            current = root
    return "".join(result)

def check_prefix_property(codes: dict) → bool:
    code_list = sorted(codes.values())
    for i in range(len(code_list) - 1):
        if code_list[i + 1].startswith(code_list[i]):

```

```

        return False
    return True

def print_complexity():
    print("\n≡≡≡ Оценка сложности алгоритма ≡≡≡")
    print("  n – количество различных символов в тексте")
    print("  Подсчёт частот:          O(N), где N – длина текста")
    print("  Построение дерева:       O(n log n) – n-1 операций над кучей")
    print("  Построение таблицы:      O(n)")
    print("  Кодирование текста:      O(N * L), L – средняя длина кода ≈ log n")
    print("  Декодирование текста:    O(M), M – длина закодированной строки")
    print("  Итого (доминирует):      O(N log n)")

def main():
    text = input("Введите текст для кодирования: ")
    if not text:
        text = "hello"

    print(f"\nИсходный текст: «{text}»")
    print(f"Длина текста: {len(text)} символов")

    start = time.perf_counter()

    freq = count_frequencies(text)
    print("\nЧастоты символов")
    for ch, f in sorted(freq.items(), key=lambda x: -x[1]):
        label = repr(ch)
        print(f"  {label:6s}: {f}")

    root = build_huffman_tree(freq)

    codes = build_codes(root)
    print("\nСловарь Хаффмана (условие Фано выполнено)")
    for ch, code in sorted(codes.items(), key=lambda x: len(x[1])):
        label = repr(ch)
        print(f"  {label:6s}: {code}")

    ok = check_prefix_property(codes)

```

```

    print(f"\nПрефиксное свойство (условие Фано): {'✓ выполнено' if ok else
'✗ нарушено'}")

    encoded = encode(text, codes)
    print(f"\nКодирование")
    print(f"Закодированная строка ({len(encoded)} бит):")
    print(f"  {encoded}")

    decoded = decode(encoded, root)
    print(f"\nДекодирование")
    print(f"Восстановленный текст: «{decoded}»")
    print(f"Декодирование корректно: {'✓' if decoded == text else '✗'}")

    end = time.perf_counter()

    original_bits = len(text) * 8
    huffman_bits = len(encoded)
    savings = (1 - huffman_bits / original_bits) * 100

    print(f"\n-Сравнение размеров")
    print(f"  Без сжатия (8 бит/символ): {original_bits} бит")
    print(f"  Код Хаффмана:                {huffman_bits} бит")
    print(f"  Экономия:                        {savings:.1f}%")

    print(f"\nВремя выполнения: {(end - start) * 1000:.4f} мс")

    print_complexity()

if __name__ == "__main__":
    main()

```

Пример выполнения:

Введите текст для кодирования: привет

Исходный текст: «привет»

Длина текста: 6 символов

Частоты символов

'п' : 1

'р' : 1

'и' : 1

'в' : 1

'е' : 1

'т' : 1

Словарь Хаффмана (условие Фано выполнено)

'е' : 00

'в' : 01

'п' : 100

'р' : 101

'т' : 110

'и' : 111

Префиксное свойство (условие Фано): ✓ выполнено

Кодирование

Закодированная строка (16 бит):

1001011110100110

Декодирование

Восстановленный текст: «привет»

Декодирование корректно: ✓

–Сравнение размеров

Без сжатия (8 бит/символ): 48 бит

Код Хаффмана: 16 бит

Экономия: 66.7%

Время выполнения: 0.1227 мс

=== Оценка сложности алгоритма ===

n – количество различных символов в тексте

Подсчёт частот: $O(N)$, где N – длина текста

Построение дерева: $O(n \log n)$ – n-1 операций над кучей

Построение таблицы: $O(n)$

Кодирование текста: $O(N * L)$, L – средняя длина кода $\approx \log n$

Декодирование текста: $O(M)$, M – длина закодированной строки

Итого (доминирует): $O(N \log n)$

Рисунок 20 – Пример выполнения десятого задания